

MyMusicTaste

0.1.0

Generated by Doxygen 1.14.0

1 Namespace Index	1
1.1 Namespace List	1
2 Hierarchical Index	3
2.1 Class Hierarchy	3
3 Class Index	5
3.1 Class List	5
4 Namespace Documentation	9
4.1 MyMusicTaste Namespace Reference	9
4.2 MyMusicTaste.Components Namespace Reference	9
4.3 MyMusicTaste.Components.Comments Namespace Reference	9
4.4 MyMusicTaste.Components.Dialogs Namespace Reference	9
4.5 MyMusicTaste.Components.Misc Namespace Reference	10
4.6 MyMusicTaste.Components.Page_Auth Namespace Reference	10
4.7 MyMusicTaste.Components.Page_Home Namespace Reference	10
4.8 MyMusicTaste.Components.Page_Search Namespace Reference	10
4.9 MyMusicTaste.Components.Page_Song Namespace Reference	10
4.10 MyMusicTaste.Components.Page_SongSubmission Namespace Reference	11
4.11 MyMusicTaste.Components.Page_User Namespace Reference	11
4.12 MyMusicTaste.Database Namespace Reference	11
4.13 MyMusicTaste.Database.Contexts Namespace Reference	11
4.14 MyMusicTaste.Database.Contexts.MongoDb Namespace Reference	11
4.15 MyMusicTaste.Database.Contexts.MongoDb.Operations Namespace Reference	12
4.16 MyMusicTaste.Database.Operations Namespace Reference	12
4.17 MyMusicTaste.Models Namespace Reference	13
4.18 MyMusicTaste.Startup Namespace Reference	13
4.19 MyMusicTaste.Utils Namespace Reference	13
4.19.1 Function Documentation	13
4.19.1.1 CssSize()	13
5 Class Documentation	15
5.1 AuthController Class Reference	15
5.1.1 Member Function Documentation	15
5.1.1.1 LoginAsync()	15
5.1.1.2 LogoutAsync()	16
5.2 Comment Class Reference	16
5.3 CommentComp Class Reference	17
5.4 CommentSection Class Reference	17
5.5 ConfirmDialog Class Reference	18
5.6 DatabaseOperationException Class Reference	19
5.7 Dialog< TResult > Class Template Reference	20
5.7.1 Detailed Description	21

5.7.2 Member Function Documentation	21
5.7.2.1 OpenFileDialog()	21
5.8 DialogBase Class Reference	21
5.9 DialogResult< TResult > Struct Template Reference	22
5.9.1 Detailed Description	22
5.10 Dropzone Class Reference	22
5.10.1 Member Function Documentation	23
5.10.1.1 SetActive()	23
5.11 Histogram< TValue, TLabel > Class Template Reference	23
5.11.1 Detailed Description	23
5.12 ICommentsListing Interface Reference	24
5.12.1 Member Function Documentation	24
5.12.1.1 GetCommentsByPageAsync()	24
5.12.1.2 GetCommentsByUserAsync()	25
5.13 IRepository< TModel > Interface Template Reference	25
5.13.1 Detailed Description	25
5.13.2 Member Function Documentation	26
5.13.2.1 CreateAsync()	26
5.13.2.2 DeleteAsync()	26
5.13.2.3 GetById() [1/2]	26
5.13.2.4 GetById() [2/2]	26
5.13.2.5 GetByIdAsync()	27
5.13.2.6 GetByIdsAsync()	27
5.13.2.7 UpdateAsync()	27
5.14 IIdentityProvider Interface Reference	28
5.14.1 Member Function Documentation	28
5.14.1.1 AssignRoleAsync()	28
5.14.1.2 AuthorizeUserById()	29
5.14.1.3 GetUserId()	29
5.14.1.4 IsAuthenticated()	29
5.14.1.5 LoginUserAsync()	29
5.14.1.6 LogoutUserAsync()	30
5.14.1.7 SignUpUserAsync()	30
5.15 InputDialog Class Reference	30
5.16 ISearchOperation< TModel > Interface Template Reference	32
5.16.1 Detailed Description	32
5.16.2 Member Function Documentation	32
5.16.2.1 SearchAsync()	32
5.17 ISongRatingListing Interface Reference	33
5.17.1 Member Function Documentation	33
5.17.1.1 GetRatingsBySongAsync()	33
5.17.1.2 GetRatingsByUserAsync()	33

5.17.1.3 GetSongRatingAsync()	34
5.18 ISongStatsCalculation Interface Reference	34
5.18.1 Member Function Documentation	34
5.18.1.1 CalculateSongStatsAsync()	34
5.19 ISongSubmission Interface Reference	35
5.19.1 Member Function Documentation	35
5.19.1.1 SubmitSongAsync()	35
5.20 IUserLoginDto Interface Reference	36
5.21 IUserSignupDto Interface Reference	36
5.22 IUserStatsCalculation Interface Reference	37
5.22.1 Member Function Documentation	37
5.22.1.1 CalculateUserStatsAsync()	37
5.23 AuthController.LoginDto Class Reference	37
5.23.1 Property Documentation	38
5.23.1.1 Password	38
5.23.1.2 Username	38
5.24 LoginPage Class Reference	38
5.25 Model Class Reference	39
5.26 MongoCommentsListing Class Reference	39
5.26.1 Member Function Documentation	39
5.26.1.1 GetCommentsByPageAsync()	39
5.26.1.2 GetCommentsByUserAsync()	40
5.27 MongoDBContext Class Reference	40
5.27.1 Member Function Documentation	40
5.27.1.1 Connect()	40
5.28 Mongoidentity Class Reference	41
5.28.1 Constructor & Destructor Documentation	42
5.28.1.1 Mongoidentity()	42
5.28.2 Member Function Documentation	42
5.28.2.1 AssignRoleAsync()	42
5.28.2.2 AuthorizeUserById()	42
5.28.2.3 Configure()	43
5.28.2.4 GetUserId()	43
5.28.2.5 IsAuthenticated()	43
5.28.2.6 LoginUserAsync()	43
5.28.2.7 LogoutUserAsync()	44
5.28.2.8 SignupUserAsync()	44
5.29 MongoRepository< TModel > Class Template Reference	44
5.29.1 Detailed Description	45
5.29.2 Member Function Documentation	45
5.29.2.1 CreateAsync()	45
5.29.2.2 DeleteAsync()	45

5.29.2.3 GetById() [1/2]	46
5.29.2.4 GetById() [2/2]	46
5.29.2.5 GetByIdAsync()	46
5.29.2.6 GetByIdsAsync()	47
5.29.2.7 UpdateAsync()	47
5.30 MongoSongRatingListing Class Reference	48
5.30.1 Member Function Documentation	48
5.30.1.1 GetRatingsBySongAsync()	48
5.30.1.2 GetRatingsByUserAsync()	48
5.30.1.3 GetSongRatingAsync()	49
5.31 MongoSongSearch Class Reference	49
5.31.1 Member Function Documentation	49
5.31.1.1 SearchAsync()	49
5.32 MongoSongsStatsCalculation Class Reference	50
5.32.1 Member Function Documentation	50
5.32.1.1 CalculateSongStatsAsync()	50
5.33 MongoUserSearch Class Reference	51
5.33.1 Member Function Documentation	51
5.33.1.1 SearchAsync()	51
5.34 MongoUserStatsCalculation Class Reference	51
5.34.1 Member Function Documentation	52
5.34.1.1 CalculateUserStatsAsync()	52
5.35 RateSongComp Class Reference	52
5.36 SearchComponent< TModel > Class Template Reference	53
5.36.1 Detailed Description	53
5.37 SearchPage Class Reference	53
5.38 Song Class Reference	54
5.39 SongPage Class Reference	54
5.40 SongRating Class Reference	55
5.41 SongRatingItem Class Reference	56
5.42 SongRatingSection Class Reference	56
5.43 SongSearchItem Class Reference	56
5.44 SongStats Class Reference	57
5.45 SongSubmission Class Reference	57
5.45.1 Member Function Documentation	57
5.45.1.1 SubmitSongAsync()	57
5.46 SongSubmissionPage Class Reference	58
5.47 Thumbnail Class Reference	58
5.48 User Class Reference	59
5.49 UserPage Class Reference	60
5.50 UserSearchItem Class Reference	60
5.51 UserSignupForm Class Reference	61

5.52 UserStats Class Reference	61
Index	63

Chapter 1

Namespace Index

1.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

MyMusicTaste	9
MyMusicTaste.Components	9
MyMusicTaste.Components.Comments	9
MyMusicTaste.Components.Dialogs	9
MyMusicTaste.Components.Misc	10
MyMusicTaste.Components.Page_Auth	10
MyMusicTaste.Components.Page_Home	10
MyMusicTaste.Components.Page_Search	10
MyMusicTaste.Components.Page_Song	10
MyMusicTaste.Components.Page_SongSubmission	11
MyMusicTaste.Components.Page_User	11
MyMusicTaste.Database	11
MyMusicTaste.Database.Contexts	11
MyMusicTaste.Database.Contexts.MongoDb	11
MyMusicTaste.Database.Contexts.MongoDb.Operations	12
MyMusicTaste.Database.Operations	12
MyMusicTaste.Models	13
MyMusicTaste.Startup	13
MyMusicTaste.Utils	13

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

AuthController	15
CommentComp	17
CommentSection	17
DatabaseOperationException	19
Dialog< TResult >	20
ConfirmDialog	18
InputDialog	30
DialogBase	21
DialogResult< TResult >	22
Dropzone	22
Histogram< TValue, TLabel >	23
ICommentsListing	24
MongoCommentsListing	39
IDbRepository< TModel >	25
MongoRepository< TModel >	44
IIdentityProvider	28
Mongoidentity	41
ISearchOperation< TModel >	32
MongoSongSearch	49
MongoUserSearch	51
ISongRatingListing	33
MongoSongRatingListing	48
ISongStatsCalculation	34
MongoSongsStatsCalculation	50
ISongSubmission	35
SongSubmission	57
IUserLoginDto	36
AuthController.LoginDto	37
IUserSignupDto	36
IUserStatsCalculation	37
MongoUserStatsCalculation	51
LoginPage	38

Model	39
Comment	16
Song	54
SongRating	55
User	59
MongoDbContext	40
RateSongComp	52
SearchComponent< TModel >	53
SearchPage	53
SongPage	54
SongRatingItem	56
SongRatingSection	56
SongSearchItem	56
SongStats	57
SongSubmissionPage	58
Thumbnail	58
UserPage	60
UserSearchItem	60
UserSignupForm	61
UserStats	61

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

AuthController	Controller responsible for authentication operations	15
Comment	Represents a comment posted by a user on a certain page	16
CommentComp	A component that displays and manages a single comment, including editing and deletion . . .	17
CommentSection	A component that displays a list of comments for a given page and allows signed-in users to post a new comment	17
ConfirmDialog	Dialog component for simple yes/no confirmation	18
DatabaseOperationException	Represents errors that occur during general database operations	19
Dialog< TResult >	Base abstract class for all dialogs. Manages opening, closing, and confirmation/cancellation .	20
DialogBase	Provides a HTML wrapper for dialogs with header, body, and footer containing confirm/cancel buttons	21
DialogResult< TResult >	Represents the result of a dialog	22
Dropzone	A component that represents a drag-and-drop zone for items. Dropzone must be activated before it can receive items	22
Histogram< TValue, TLabel >	A generic component that renders a histogram chart based on numeric values and associated labels	23
ICommentsListing	Provides methods for retrieving comments from the database	24
IDbRepository< TModel >	Represents a generic repository for performing CRUD operations on models	25
IIdentityProvider	Provides an abstraction for user authentication, signup, login, logout, and role management . .	28
InputDialog	Dialog component that allows user to input a string	30
ISearchOperation< TModel >	Provides a generic interface for searching entities of type <i>TModel</i>	32

ISongRatingListing	Provides methods for retrieving song ratings from the database	33
ISongStatsCalculation	Provides methods for calculating statistics for a song, such as average rating, median rating, total listens, and rating distribution	34
ISongSubmission	Provides an abstraction for submitting new songs to the database while avoiding duplicates . .	35
IUserLoginDto	Data required to log in a user	36
IUserSignupDto	Data required to sign up a user	36
IUserStatsCalculation	Provides methods for calculating statistics for a user based on their song ratings	37
AuthController.LoginDto	DTO used for login requests	37
LoginPage	Page for users to login or signup	38
Model	Base class for all entities in the system, providing a unique identifier	39
MongoCommentsListing	Provides methods for retrieving comments from the MongoDb database	39
MongoDbContext	Provides a MongoDB client and a method to establish a connection	40
MongoIdentity	A MongoDB-based implementation of IIdentityProvider using ASP.NET Identity. Handles user signup, login, logout, and authentication/authorization checks	41
MongoRepository< TModel >	A MongoDB-based repository for performing CRUD operations on models	44
MongoSongRatingListing	Retrieves song ratings from a MongoDB collection	48
MongoSongSearch	Performs search operations for users in MongoDB	49
MongoSongsStatsCalculation	Calculates statistics for a song, including average rating, median rating, total listens, and rating distribution in MongoDB	50
MongoUserSearch	Performs search operations for songs in MongoDB	51
MongoUserStatsCalculation	Calculates statistics for a user based on their song ratings in MongoDB	51
RateSongComp	A component that allows a signed-in user to rate a song, view their existing rating, update it, or remove it from their collection	52
SearchComponent< TModel >	Base component for search pages. Provides shared search functionality for a given model type	53
SearchPage	A page providing a search interface for songs and users with tabbed navigation	53
Song	A model representing basic song information	54
SongPage	DPage for displaying detailed information about a song, including its metadata, statistics, and comments	54
SongRating	Represents a rating given by a user to a song	55
SongRatingItem	Displays a single song rating item for a user. Can optionally allow inline editing of the rating . .	56
SongRatingSection	Displays a list of a user's song ratings in descending order. Allows drag-and-drop to reorder ratings if the user is authorized	56

SongSearchItem	Displays an item in search results that navigates to the song's page when clicked	56
SongStats	Represents statistical information about a song, including ratings and listens	57
SongSubmission	Handles submission of new songs to the MongoDB database, ensuring no duplicates exist . .	57
SongSubmissionPage	Page for submitting a new song to the database. Validates user input and handles navigation after submission	58
Thumbnail	A component that displays an image thumbnail or a placeholder if the image link is invalid . . .	58
User	A model representing basic user information	59
UserPage	Displays a user's profile including general info, statistics, and comments. Supports editing the profile for the authorized user	60
UserSearchItem	Displays an item in search results that navigates to the user's page when clicked	60
UserSignupForm	A form component that allows new users to sign up for the application. Handles input validation, submission, and automatic login after successful registration	61
UserStats	Represents statistical information about a user's ratings	61

Chapter 4

Namespace Documentation

4.1 MyMusicTaste Namespace Reference

4.2 MyMusicTaste.Components Namespace Reference

4.3 MyMusicTaste.Components.Comments Namespace Reference

Classes

- class [CommentComp](#)
A component that displays and manages a single comment, including editing and deletion.
- class [CommentSection](#)
A component that displays a list of comments for a given page and allows signed-in users to post a new comment.

4.4 MyMusicTaste.Components.Dialogs Namespace Reference

Classes

- class [ConfirmDialog](#)
Dialog component for simple yes/no confirmation.
- class [Dialog< TResult >](#)
Base abstract class for all dialogs. Manages opening, closing, and confirmation/cancellation.
- class [DialogBase](#)
Provides a HTML wrapper for dialogs with header, body, and footer containing confirm/cancel buttons.
- struct [DialogResult< TResult >](#)
Represents the result of a dialog.
- class [InputDialog](#)
Dialog component that allows user to input a string.

4.5 MyMusicTaste.Components.Misc Namespace Reference

Classes

- class [Dropzone](#)
A component that represents a drag-and-drop zone for items. [Dropzone](#) must be activated before it can receive items.
- class [Histogram< TValue, TLabel >](#)
A generic component that renders a histogram chart based on numeric values and associated labels.
- class [Thumbnail](#)
A component that displays an image thumbnail or a placeholder if the image link is invalid.

4.6 MyMusicTaste.Components.Page_Auth Namespace Reference

Classes

- class [AuthController](#)
Controller responsible for authentication operations.
- class [LoginPage](#)
Page for users to login or signup.
- class [UserSignupForm](#)
A form component that allows new users to sign up for the application. Handles input validation, submission, and automatic login after successful registration.

4.7 MyMusicTaste.Components.Page_Home Namespace Reference

4.8 MyMusicTaste.Components.Page_Search Namespace Reference

Classes

- class [SearchComponent< TModel >](#)
Base component for search pages. Provides shared search functionality for a given model type.
- class [SearchPage](#)
A page providing a search interface for songs and users with tabbed navigation.
- class [SongSearchItem](#)
Displays an item in search results that navigates to the song's page when clicked.
- class [UserSearchItem](#)
Displays an item in search results that navigates to the user's page when clicked.

4.9 MyMusicTaste.Components.Page_Song Namespace Reference

Classes

- class [RateSongComp](#)
A component that allows a signed-in user to rate a song, view their existing rating, update it, or remove it from their collection.
- class [SongPage](#)
DPage for displaying detailed information about a song, including its metadata, statistics, and comments.

4.10 MyMusicTaste.Components.Page_SongSubmission Namespace Reference

Classes

- class [SongSubmissionPage](#)
Page for submitting a new song to the database. Validates user input and handles navigation after submission.

4.11 MyMusicTaste.Components.Page_User Namespace Reference

Classes

- class [SongRatingItem](#)
Displays a single song rating item for a user. Can optionally allow inline editing of the rating.
- class [SongRatingSection](#)
Displays a list of a user's song ratings in descending order. Allows drag-and-drop to reorder ratings if the user is authorized.
- class [UserPage](#)
Displays a user's profile including general info, statistics, and comments. Supports editing the profile for the authorized user.

4.12 MyMusicTaste.Database Namespace Reference

Classes

- interface [IDbRepository< TModel >](#)
Represents a generic repository for performing CRUD operations on models.
- interface [IIdentityProvider](#)
Provides an abstraction for user authentication, signup, login, logout, and role management.
- interface [IUserLoginDto](#)
Data required to log in a user.
- interface [IUserSignupDto](#)
Data required to sign up a user.

4.13 MyMusicTaste.Database.Contexts Namespace Reference

4.14 MyMusicTaste.Database.Contexts.MongoDb Namespace Reference

Classes

- class [MongoDbContext](#)
Provides a MongoDB client and a method to establish a connection.
- class [MongolIdentity](#)
A MongoDB-based implementation of [IIdentityProvider](#) using ASP.NET Identity. Handles user signup, login, logout, and authentication/authorization checks.
- class [MongoRepository< TModel >](#)
A MongoDB-based repository for performing CRUD operations on models.

4.15 MyMusicTaste.Database.Contexts.MongoDb.Operations Namespace Reference

Classes

- class [MongoCommentsListing](#)
Provides methods for retrieving comments from the [MongoDb](#) database.
- class [MongoSongRatingListing](#)
Retrieves song ratings from a MongoDB collection.
- class [MongoSongSearch](#)
Performs search operations for users in MongoDB.
- class [MongoSongsStatsCalculation](#)
Calculates statistics for a song, including average rating, median rating, total listens, and rating distribution in MongoDB.
- class [MongoUserSearch](#)
Performs search operations for songs in MongoDB.
- class [MongoUserStatsCalculation](#)
Calculates statistics for a user based on their song ratings in MongoDB.
- class [SongSubmission](#)
Handles submission of new songs to the MongoDB database, ensuring no duplicates exist.

4.16 MyMusicTaste.Database.Operations Namespace Reference

Classes

- class [DatabaseOperationException](#)
Represents errors that occur during general database operations.
- interface [ICommentsListing](#)
Provides methods for retrieving comments from the database.
- interface [ISearchOperation< TModel >](#)
*Provides a generic interface for searching entities of type *TModel* .*
- interface [ISongRatingListing](#)
Provides methods for retrieving song ratings from the database.
- interface [ISongStatsCalculation](#)
Provides methods for calculating statistics for a song, such as average rating, median rating, total listens, and rating distribution.
- interface [ISongSubmission](#)
Provides an abstraction for submitting new songs to the database while avoiding duplicates.
- interface [IUserStatsCalculation](#)
Provides methods for calculating statistics for a user based on their song ratings.

4.17 MyMusicTaste.Models Namespace Reference

Classes

- class [Comment](#)
Represents a comment posted by a user on a certain page.
- class [Model](#)
Base class for all entities in the system, providing a unique identifier.
- class [Song](#)
A model representing basic song information.
- class [SongRating](#)
Represents a rating given by a user to a song.
- class [SongStats](#)
Represents statistical information about a song, including ratings and listens.
- class [User](#)
A model representing basic user information.
- class [UserStats](#)
Represents statistical information about a user's ratings.

Enumerations

- enum [CommentPageType](#) { [Song](#) , [User](#) }
Types of pages that a comment can be associated with.

4.18 MyMusicTaste.Startup Namespace Reference

4.19 MyMusicTaste.Utils Namespace Reference

Enumerations

- enum [CssUnit](#) {
 [Zero](#) , [Px](#) , [Em](#) , [Rem](#) ,
 [Vh](#) , [Vw](#) , [Perc](#) }
Represents supported CSS size units.

Functions

- readonly record struct [CssSize](#) (double Value, [CssUnit](#) Unit)
Represents a CSS size consisting of a numeric value and a unit. ///.

4.19.1 Function Documentation

4.19.1.1 [CssSize\(\)](#)

```
readonly record struct CssSize (  
    double Value,  
    CssUnit Unit)
```

Parameters

<i>Value</i>	The numeric value of the size.
<i>Unit</i>	The CSS unit of the size.

Converts the size to a CSS value (e.g. "10px", "50%").

Returns

A string representation of the CSS value.

Adds a number to the size value.

Adds a number to the size value.

Adds two sizes with the same unit.

Exceptions

<i>ArgumentException</i>	Thrown when units do not match.
--------------------------	---------------------------------

Multiplies the size value by a number.

Multiplies the size value by a number.

Multiplies two sizes with the same unit.

Exceptions

<i>ArgumentException</i>	Thrown when units do not match.
--------------------------	---------------------------------

Chapter 5

Class Documentation

5.1 AuthController Class Reference

Controller responsible for authentication operations.

Classes

- class [LoginDto](#)
DTO used for login requests.

Public Member Functions

- **AuthController** ([IIdentityProvider](#) identity)
- async Task< IActionResult > [LoginAsync](#) ([FromForm] [LoginDto](#) loginDto)
Logs in a user using provided credentials. Returns a redirect to the home page on success, or Unauthorized/BadRequest on failure.
- async Task< IActionResult > [LogoutAsync](#) ()
Logs out the currently authenticated user and redirects to the home page.

Static Public Attributes

- const string **LOGIN_ROUTE** = "auth/login"
- const string **LOGOUT_ROUTE** = "auth/logout"

5.1.1 Member Function Documentation

5.1.1.1 LoginAsync()

```
async Task< IActionResult > LoginAsync (  
    [FromForm] LoginDto loginDto) [inline]
```

Parameters

<i>loginDto</i>	Login credentials.
-----------------	--------------------

Returns

An action result representing success or failure of the login attempt.

5.1.1.2 LogoutAsync()

```
async Task< IActionResult > LogoutAsync () [inline]
```

Returns

An action result redirecting to the homepage.

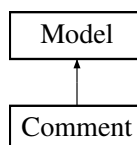
The documentation for this class was generated from the following file:

- MyMusicTaste/Components/Page-Auth/AuthController.cs

5.2 Comment Class Reference

Represents a comment posted by a user on a certain page.

Inheritance diagram for Comment:

**Properties**

- required ObjectId **UserId** [get, set]
The ID of the user who made the comment.
- required [CommentPageType](#) **CommentPageType** [get, set]
The type of page the comment was posted in.
- required ObjectId **PageId** [get, set]
The ID of the page the comment was posted to.
- string **Content** = string.Empty [get, set]
- DateTime **CreationTime** = INVALID_DATE [get, set]
- DateTime **LastEditTime** = INVALID_DATE [get, set]
- bool **CreationTimeValid** [get]
- bool **LastEditTimeValid** [get]

Properties inherited from [Model](#)

- ObjectId **Id** [get, set]

The documentation for this class was generated from the following file:

- MyMusicTaste/Models/Comment.cs

5.3 CommentComp Class Reference

A component that displays and manages a single comment, including editing and deletion.

Protected Member Functions

- override async Task **OnInitializedAsync** ()

Properties

- [Comment](#) **Comment** = null! [get, set]
The comment object to display and manage.
- bool **IsNewComment** = true [get, set]
Indicates whether this comment is newly created and not yet persisted.
- bool **IsPosted** [get]
True if the comment has already been posted to the database.
- EventCallback **OnEditModeOpened** [get, set]
Event triggered when the edit mode is opened.
- EventCallback< bool > **OnEditModeClosed** [get, set]
Event triggered when the edit mode is closed. The callback receives true if changes were saved, false otherwise.
- EventCallback **OnChangesSaved** [get, set]
Event triggered after changes to the comment have been saved.
- EventCallback **OnCommentDeleted** [get, set]
Event triggered after the comment has been deleted.

The documentation for this class was generated from the following file:

- MyMusicTaste/Components/Comments/CommentComp.razor.cs

5.4 CommentSection Class Reference

A component that displays a list of comments for a given page and allows signed-in users to post a new comment.

Protected Member Functions

- override async Task **OnInitializedAsync** ()

Properties

- [CommentPageType](#) **PageType** [get, set]
The type of page the comments was posted to.
- string **PageId** = null! [get, set]
The ID of the page for which to display comments.
- int **ResultsCount** [get, set]
The maximum number of comments to fetch and display.

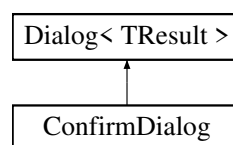
The documentation for this class was generated from the following file:

- MyMusicTaste/Components/Comments/CommentSection.razor.cs

5.5 ConfirmDialog Class Reference

Dialog component for simple yes/no confirmation.

Inheritance diagram for ConfirmDialog:



Protected Member Functions

- override bool **GetResult** ()
Returns the true if the dialog has been confirmed.

Protected Member Functions inherited from [Dialog< TResult >](#)

- override void **OnAfterRender** (bool firstRender)
- TResult **GetResult** ()
Returns the result of the dialog when confirmed.

Properties

- string **Question** = null! [get, set]
Question text to display in the dialog.

Properties inherited from [Dialog< TResult >](#)

- string? **Title** [get, set]
Title of the dialog.
- string **ConfirmText** = "Confirm" [get, set]
Text for the confirm button.
- string **CancelText** = "Cancel" [get, set]
Text for the cancel button.
- [DialogBase](#) **BaseDialog** = null! [get, set]
Reference to the underlying HTML wrapper dialog component.
- TaskCompletionSource< bool > **CompletionToken** = new() [get]
Token used to track dialog completion.
- Task< bool > **CompletionTask** [get]
Task representing the completion of the dialog.

Additional Inherited Members**Public Member Functions inherited from [Dialog< TResult >](#)**

- async Task< DialogResult< TResult > > [OpenDialog](#) ()
Opens the dialog and returns a [DialogResult< TResult>](#) when confirmed or canceled.
- void **Confirm** ()
Marks the dialog as confirmed.
- void **Cancel** ()
Marks the dialog as canceled.

The documentation for this class was generated from the following file:

- MyMusicTaste/Components/Dialogs/ConfirmDialog.razor.cs

5.6 DatabaseOperationException Class Reference

Represents errors that occur during general database operations.

Public Member Functions

- **DatabaseOperationException** (string message)
- **DatabaseOperationException** (string message, Exception innerException)

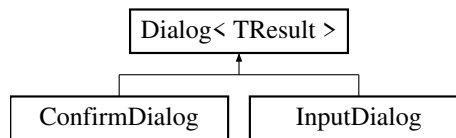
The documentation for this class was generated from the following file:

- MyMusicTaste/Database/Operations/DatabaseOperationException.cs

5.7 Dialog< TResult > Class Template Reference

Base abstract class for all dialogs. Manages opening, closing, and confirmation/cancellation.

Inheritance diagram for Dialog< TResult >:



Public Member Functions

- `async Task< DialogResult< TResult > > OpenDialog ()`
Opens the dialog and returns a [DialogResult< TResult>](#) when confirmed or canceled.
- `void Confirm ()`
Marks the dialog as confirmed.
- `void Cancel ()`
Marks the dialog as canceled.

Protected Member Functions

- override `void OnAfterRender (bool firstRender)`
- `TResult GetResult ()`
Returns the result of the dialog when confirmed.

Properties

- `string? Title [get, set]`
Title of the dialog.
- `string ConfirmText = "Confirm" [get, set]`
Text for the confirm button.
- `string CancelText = "Cancel" [get, set]`
Text for the cancel button.
- `DialogBase BaseDialog = null! [get, set]`
Reference to the underlying HTML wrapper dialog component.
- `TaskCompletionSource< bool > CompletionToken = new() [get]`
Token used to track dialog completion.
- `Task< bool > CompletionTask [get]`
Task representing the completion of the dialog.

5.7.1 Detailed Description

Template Parameters

<i>TResult</i>	The type of the result returned when the dialog is confirmed.
----------------	---

5.7.2 Member Function Documentation

5.7.2.1 OpenFileDialog()

```
async Task< DialogResult< TResult > > OpenFileDialog () [inline]
```

Returns

A task for the result of the dialog

The documentation for this class was generated from the following file:

- MyMusicTaste/Components/Dialogs/Dialog.cs

5.8 DialogBase Class Reference

Provides a HTML wrapper for dialogs with header, body, and footer containing confirm/cancel buttons.

Public Member Functions

- void **Open** ()
Makes the dialog visible.
- void **Close** ()
Makes the dialog invisible.

Properties

- RenderFragment **BodyContent** = null! [get, set]
Content to display inside the dialog body.
- string? **Title** [get, set]
Title of the dialog.
- string **ConfirmText** = "Yes" [get, set]
Text for the confirm button.
- string **CancelText** = "No" [get, set]
Text for the cancel button.
- bool **IsVisible** [get]
Indicates whether the dialog is currently visible.

Events

- Action? **ConfirmPressed**
Event triggered when the confirm button is pressed.
- Action? **CancelPressed**
Event triggered when the cancel button is pressed.

The documentation for this class was generated from the following file:

- MyMusicTaste/Components/Dialogs/DialogBase.razor.cs

5.9 DialogResult< TResult > Struct Template Reference

Represents the result of a dialog.

Properties

- bool **WasConfirmed** [get, set]
Indicates if the dialog was confirmed.
- TResult? **Result** [get, set]
The value returned if the dialog was confirmed; otherwise default.

5.9.1 Detailed Description

Template Parameters

<i>TResult</i>	Type of the value returned when confirmed.
----------------	--

The documentation for this struct was generated from the following file:

- MyMusicTaste/Components/Dialogs/DialogResult.cs

5.10 Dropzone Class Reference

A component that represents a drag-and-drop zone for items. [Dropzone](#) must be activated before it can receive items.

Public Member Functions

- void [SetActive](#) (bool active)
Sets the active state of the dropzone.

Properties

- bool **AutoActivate** = true [get, set]
If true, the dropzone automatically activates when an item is dragged over it.
- bool **AutoDeactivate** = true [get, set]
If true, the dropzone automatically deactivates when an item leaves it.
- EventCallback< Dropzone > **OnDragEnter** [get, set]
Event triggered when an item is dragged into the dropzone.
- EventCallback< Dropzone > **OnDragLeave** [get, set]
Event triggered when an item leaves the dropzone.
- EventCallback< Dropzone > **OnDrop** [get, set]
Event triggered when an item is dropped onto the dropzone.

5.10.1 Member Function Documentation

5.10.1.1 SetActive()

```
void SetActive (
    bool active) [inline]
```

Parameters

<i>active</i>	True to activate, false to deactivate.
---------------	--

The documentation for this class was generated from the following file:

- MyMusicTaste/Components/Misc/Dropzone.razor.cs

5.11 Histogram< TValue, TLabel > Class Template Reference

A generic component that renders a histogram chart based on numeric values and associated labels.

Protected Member Functions

- override void **OnInitialized** ()

Properties

- TValue[] **Values** = null! [get, set]
Array of numeric values to display in the histogram.
- TLabel[] **Labels** = null! [get, set]
Array of labels corresponding to the values.
- CssSize **MaxHeight** [get, set]
Maximum height of the histogram bars.

5.11.1 Detailed Description

Template Parameters

<i>TValue</i>	The numeric type of the values to display.
<i>TLabel</i>	The type of labels for each value.

Type Constraints

***TValue* : *INumber*<*TValue*>**

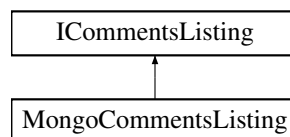
The documentation for this class was generated from the following file:

- MyMusicTaste/Components/Misc/Histogram.razor.cs

5.12 ICommentsListing Interface Reference

Provides methods for retrieving comments from the database.

Inheritance diagram for ICommentsListing:



Public Member Functions

- Task< IEnumerable< [Comment](#) > > [GetCommentsByUserAsync](#) (string userId, int resultCount)
Retrieves comments made by a specific user.
- Task< IEnumerable< [Comment](#) > > [GetCommentsByPageAsync](#) ([CommentPageType](#) pageType, string pageId, int resultCount)
Retrieves comments associated with a specific page, ordered by creation time descending.

5.12.1 Member Function Documentation

5.12.1.1 GetCommentsByPageAsync()

```

Task< IEnumerable< Comment > > GetCommentsByPageAsync (
    CommentPageType pageType,
    string pageId,
    int resultCount)
  
```

Parameters

<i>pageType</i>	The type of the page the comments belong to.
<i>pageId</i>	The ID of the page.
<i>resultCount</i>	The maximum number of comments to return.

Returns

A task for the list of comments.

Implemented in [MongoCommentsListing](#).

5.12.1.2 GetCommentsByUserAsync()

```
Task< IEnumerable< Comment > > GetCommentsByUserAsync (
    string userId,
    int resultCount)
```

Parameters

<i>userId</i>	The ID of the user whose comments to retrieve.
<i>resultCount</i>	The maximum number of comments to return.

Returns

A task for the collection of comments.

Implemented in [MongoCommentsListing](#).

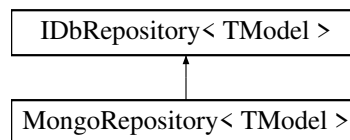
The documentation for this interface was generated from the following file:

- MyMusicTaste/Database/Operations/ICommentsListing.cs

5.13 IDbRepository< TModel > Interface Template Reference

Represents a generic repository for performing CRUD operations on models.

Inheritance diagram for IDbRepository< TModel >:



Public Member Functions

- TModel [GetById](#) (string? id)
Retrieves a model by its ID from the database.
- TModel [GetById](#) (ObjectId id)
Retrieves a model by its ID from the database.
- Task< TModel > [GetByIdAsync](#) (string? id)
Retrieves a model by its string ID from the database.
- Task< IEnumerable< TModel > > [GetByIdsAsync](#) (IEnumerable< string > ids)
Asynchronously retrieves multiple models by their IDs from the database.
- Task [CreateAsync](#) (TModel model)
Asynchronously inserts a new model to the database.
- Task [UpdateAsync](#) (TModel model)
Asynchronously updates an existing model in the database.
- Task [DeleteAsync](#) (TModel model)
Asynchronously deletes a model from the database.

5.13.1 Detailed Description

Template Parameters

<i>TModel</i>	The type of model managed by this repository. Must inherit from Model .
---------------	---

5.13.2 Member Function Documentation

5.13.2.1 CreateAsync()

```
Task CreateAsync (
    TModel model)
```

Parameters

<i>model</i>	The model to insert.
--------------	----------------------

Implemented in [MongoRepository< TModel >](#).

5.13.2.2 DeleteAsync()

```
Task DeleteAsync (
    TModel model)
```

Parameters

<i>model</i>	The model to delete.
--------------	----------------------

Implemented in [MongoRepository< TModel >](#).

5.13.2.3 GetById() [1/2]

```
TModel GetById (
    ObjectId id)
```

Parameters

<i>id</i>	The unique identifier of the model.
-----------	-------------------------------------

Returns

The model corresponding to the specified ID.

Exceptions

<i>EntryNotFoundException</i>	Thrown when the entry does not exist.
-------------------------------	---------------------------------------

Implemented in [MongoRepository< TModel >](#).

5.13.2.4 GetById() [2/2]

```
TModel GetById (
    string? id)
```

Parameters

<i>id</i>	The unique identifier of the model.
-----------	-------------------------------------

Returns

The model corresponding to the specified ID.

Exceptions

<i>EntryNotFoundException</i>	Thrown when the entry does not exist.
-------------------------------	---------------------------------------

Implemented in [MongoRepository< TModel >](#).

5.13.2.5 GetByIdAsync()

```
Task< TModel > GetByIdAsync (
    string? id)
```

Parameters

<i>id</i>	The unique identifier of the model.
-----------	-------------------------------------

Returns

A task for the retrieved model.

Exceptions

<i>EntryNotFoundException</i>	Thrown when the entry does not exist.
-------------------------------	---------------------------------------

Implemented in [MongoRepository< TModel >](#).

5.13.2.6 GetByIdsAsync()

```
Task< IEnumerable< TModel > > GetByIdsAsync (
    IEnumerable< string > ids)
```

Parameters

<i>ids</i>	The collection of unique identifiers to retrieve.
------------	---

Returns

A task for the collection of retrieved models.

Implemented in [MongoRepository< TModel >](#).

5.13.2.7 UpdateAsync()

```
Task UpdateAsync (
    TModel model)
```

Parameters

<i>model</i>	The model with updated values.
--------------	--------------------------------

Implemented in [MongoRepository< TModel >](#).

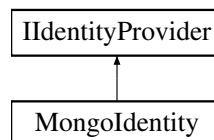
The documentation for this interface was generated from the following file:

- MyMusicTaste/Database/IDbRepository.cs

5.14 IIdentityProvider Interface Reference

Provides an abstraction for user authentication, signup, login, logout, and role management.

Inheritance diagram for IIdentityProvider:



Public Member Functions

- Task< IdentityResult > [SignUpUserAsync](#) (IUserSignupDto userSignup)
Signs up a new user.
- Task< SignInResult > [LoginUserAsync](#) (IUserLoginDto userLoginDto)
Logs in a user.
- Task [LogOutUserAsync](#) ()
Logs out the currently signed-in user.
- Task< IdentityResult > [AssignRoleAsync](#) (IdentityUser user, IdentityRole role)
Assigns a role to a user.
- bool [IsAuthenticated](#) ()
Checks if a user is authenticated.
- bool [AuthorizeUserById](#) (string requiredUserId)
Checks if a signed-in user has a specific ID.
- string? [GetUserId](#) ()
Gets the currently signed-in user's ID.

5.14.1 Member Function Documentation

5.14.1.1 AssignRoleAsync()

```

Task< IdentityResult > AssignRoleAsync (
    IdentityUser user,
    IdentityRole role)
  
```

Parameters

<i>user</i>	The user to assign the role to.
<i>role</i>	The role to assign.

Returns

A task for the IdentityResult.

Implemented in [MongolIdentity](#).

5.14.1.2 AuthorizeUserById()

```
bool AuthorizeUserById (  
    string requiredUserId)
```

Parameters

<i>required</i> ↔ <i>UserId</i>	The required user ID.
------------------------------------	-----------------------

Returns

True if the user has the specified ID; otherwise false.

Implemented in [MongolIdentity](#).

5.14.1.3 GetUserId()

```
string? GetUserId ()
```

Returns

The user ID as a string, or null if not authenticated.

Implemented in [MongolIdentity](#).

5.14.1.4 IsAuthenticated()

```
bool IsAuthenticated ()
```

Returns

True if a user is authenticated; otherwise false.

Implemented in [MongolIdentity](#).

5.14.1.5 LoginUserAsync()

```
Task< SignInResult > LoginUserAsync (  
    IUserLoginDto userLoginDto)
```

Parameters

<i>userLoginDto</i>	The user login data.
---------------------	----------------------

Returns

A task for the SignInResult indicating success or failure.

Implemented in [MongolIdentity](#).

5.14.1.6 LogOutUserAsync()

```
Task LogOutUserAsync ()
```

Implemented in [MongolIdentity](#).

5.14.1.7 SignUpUserAsync()

```
Task< IdentityResult > SignUpUserAsync (  
    IUserSignupDto userSignup)
```

Parameters

<i>userSignup</i>	The user signup data.
-------------------	-----------------------

Returns

A task for the IdentityResult indicating success or failure.

Implemented in [MongolIdentity](#).

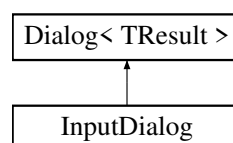
The documentation for this interface was generated from the following file:

- MyMusicTaste/Database/IIdentityProvider.cs

5.15 InputDialog Class Reference

Dialog component that allows user to input a string.

Inheritance diagram for InputDialog:



Protected Member Functions

- override string **GetResult** ()
Returns the text entered in the dialog.

Protected Member Functions inherited from [Dialog< TResult >](#)

- override void **OnAfterRender** (bool firstRender)
- TResult **GetResult** ()
Returns the result of the dialog when confirmed.

Properties

- string **Placeholder** = "" [get, set]
Placeholder text displayed inside the input area.

Properties inherited from [Dialog< TResult >](#)

- string? **Title** [get, set]
Title of the dialog.
- string **ConfirmText** = "Confirm" [get, set]
Text for the confirm button.
- string **CancelText** = "Cancel" [get, set]
Text for the cancel button.
- [DialogBase](#) **BaseDialog** = null! [get, set]
Reference to the underlying HTML wrapper dialog component.
- TaskCompletionSource< bool > **CompletionToken** = new() [get]
Token used to track dialog completion.
- Task< bool > **CompletionTask** [get]
Task representing the completion of the dialog.

Additional Inherited Members

Public Member Functions inherited from [Dialog< TResult >](#)

- async Task< DialogResult< TResult > > [OpenDialog](#) ()
Opens the dialog and returns a [DialogResult< TResult >](#) when confirmed or canceled.
- void **Confirm** ()
Marks the dialog as confirmed.
- void **Cancel** ()
Marks the dialog as canceled.

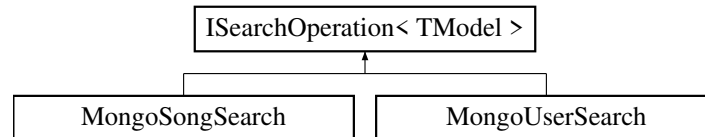
The documentation for this class was generated from the following file:

- MyMusicTaste/Components/Dialogs/InputDialog.razor.cs

5.16 ISearchOperation< TModel > Interface Template Reference

Provides a generic interface for searching entities of type *TModel* .

Inheritance diagram for ISearchOperation< TModel >:



Public Member Functions

- Task< IEnumerable< TModel >?> [SearchAsync](#) (string query, int resultsCount)
Searches entities based on a query string.

5.16.1 Detailed Description

Template Parameters

<i>TModel</i>	The type of model to search for.
---------------	----------------------------------

Type Constraints

***TModel* : Model**

5.16.2 Member Function Documentation

5.16.2.1 SearchAsync()

```
Task< IEnumerable< TModel >?> SearchAsync (
    string query,
    int resultsCount)
```

Parameters

<i>query</i>	The search query string.
<i>resultsCount</i>	The maximum number of results to return.

Returns

A task for the collection of matching entities, or null if the query is empty.

Implemented in [MongoSongSearch](#), and [MongoUserSearch](#).

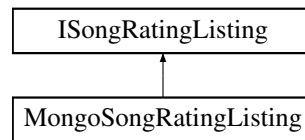
The documentation for this interface was generated from the following file:

- MyMusicTaste/Database/Operations/ISearchOperation.cs

5.17 ISongRatingListing Interface Reference

Provides methods for retrieving song ratings from the database.

Inheritance diagram for ISongRatingListing:



Public Member Functions

- Task< [SongRating](#) > [GetSongRatingAsync](#) (string songId, string userId)
Retrieves the rating a specific user has given to a specific song.
- Task< IEnumerable< [SongRating](#) > > [GetRatingsByUserAsync](#) (string userId)
Retrieves all ratings submitted by a specific user.
- Task< IEnumerable< [SongRating](#) > > [GetRatingsBySongAsync](#) (string songId)
Retrieves all ratings associated with a specific song.

5.17.1 Member Function Documentation

5.17.1.1 GetRatingsBySongAsync()

```
Task< IEnumerable< SongRating > > GetRatingsBySongAsync (
    string songId)
```

Parameters

<i>songId</i>	The ID of the song.
---------------	---------------------

Returns

A task for the list of ratings.

Implemented in [MongoSongRatingListing](#).

5.17.1.2 GetRatingsByUserAsync()

```
Task< IEnumerable< SongRating > > GetRatingsByUserAsync (
    string userId)
```

Parameters

<i>userId</i>	The ID of the user.
---------------	---------------------

Returns

A task for the list of ratings.

Implemented in [MongoSongRatingListing](#).

5.17.1.3 GetSongRatingAsync()

```
Task< SongRating > GetSongRatingAsync (
    string songId,
    string userId)
```

Parameters

<i>songId</i>	The ID of the song.
<i>userId</i>	The ID of the user.

Returns

A task for the retrieved song rating, or null if no rating exists.

Implemented in [MongoSongRatingListing](#).

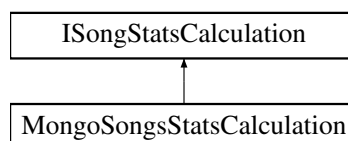
The documentation for this interface was generated from the following file:

- MyMusicTaste/Database/Operations/ISongRatingListing.cs

5.18 ISongStatsCalculation Interface Reference

Provides methods for calculating statistics for a song, such as average rating, median rating, total listens, and rating distribution.

Inheritance diagram for ISongStatsCalculation:

**Public Member Functions**

- Task< SongStats > [CalculateSongStatsAsync](#) (Song song)
Calculates statistics for a song.

5.18.1 Member Function Documentation**5.18.1.1 CalculateSongStatsAsync()**

```
Task< SongStats > CalculateSongStatsAsync (
    Song song)
```

Parameters

<i>song</i>	The song to calculate statistics for.
-------------	---------------------------------------

Returns

A task for the song's statistics.

Implemented in [MongoSongsStatsCalculation](#).

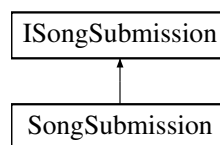
The documentation for this interface was generated from the following file:

- MyMusicTaste/Database/Operations/ISongStatsCalculation.cs

5.19 ISongSubmission Interface Reference

Provides an abstraction for submitting new songs to the database while avoiding duplicates.

Inheritance diagram for ISongSubmission:



Public Member Functions

- Task< string > [SubmitSongAsync](#) ([Song](#) songModel)
Submits a new song.

5.19.1 Member Function Documentation

5.19.1.1 SubmitSongAsync()

```
Task< string > SubmitSongAsync (  
    Song songModel)
```

Parameters

<i>songModel</i>	The song to submit.
------------------	---------------------

Returns

A task for the ID of the newly created song as a string.

Exceptions

<i>EntryAlreadyExistsException</i>	Thrown if the song already exists.
------------------------------------	------------------------------------

Implemented in [SongSubmission](#).

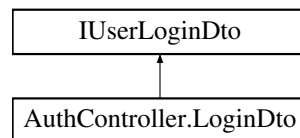
The documentation for this interface was generated from the following file:

- MyMusicTaste/Database/Operations/ISongSubmission.cs

5.20 IUserLoginDto Interface Reference

Data required to log in a user.

Inheritance diagram for IUserLoginDto:



Properties

- string **Username** [get, set]
- string **Password** [get, set]

The documentation for this interface was generated from the following file:

- MyMusicTaste/Database/IIdentityProvider.cs

5.21 IUserSignupDto Interface Reference

Data required to sign up a user.

Properties

- string **Username** [get, set]
- string **Email** [get, set]
- string **Password** [get, set]

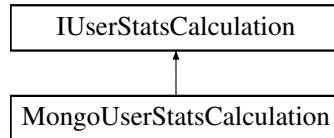
The documentation for this interface was generated from the following file:

- MyMusicTaste/Database/IIdentityProvider.cs

5.22 IUserStatsCalculation Interface Reference

Provides methods for calculating statistics for a user based on their song ratings.

Inheritance diagram for IUserStatsCalculation:



Public Member Functions

- Task< [UserStats](#) > [CalculateUserStatsAsync](#) (string userId)
Calculates statistics for a user, such as average ratings grouped by album, author, and genre.

5.22.1 Member Function Documentation

5.22.1.1 CalculateUserStatsAsync()

```
Task< UserStats > CalculateUserStatsAsync (  
    string userId)
```

Parameters

<i>userId</i>	The ID of the user to calculate statistics for.
---------------	---

Returns

A task for the user's statistics.

Implemented in [MongoUserStatsCalculation](#).

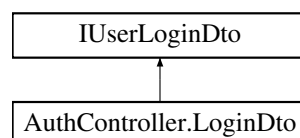
The documentation for this interface was generated from the following file:

- MyMusicTaste/Database/Operations/IUserStatsCalculation.cs

5.23 AuthController.LoginDto Class Reference

DTO used for login requests.

Inheritance diagram for AuthController.LoginDto:



Properties

- string [Username](#) = null! [get, set]
- string [Password](#) = null! [get, set]

5.23.1 Property Documentation

5.23.1.1 Password

```
string Password = null! [get], [set]
```

Implements [IUserLoginDto](#).

5.23.1.2 Username

```
string Username = null! [get], [set]
```

Implements [IUserLoginDto](#).

The documentation for this class was generated from the following file:

- MyMusicTaste/Components/Page-Auth/AuthController.cs

5.24 LoginPage Class Reference

Page for users to login or signup.

Static Public Member Functions

- static string **GetRoute** ()

Static Public Attributes

- const string **ROUTE_TEMPLATE** = "/login"

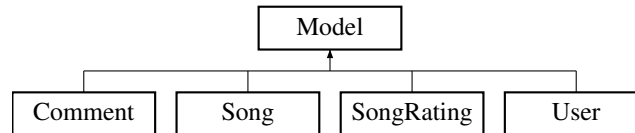
The documentation for this class was generated from the following file:

- MyMusicTaste/Components/Page-Auth/LoginPage.razor.cs

5.25 Model Class Reference

Base class for all entities in the system, providing a unique identifier.

Inheritance diagram for Model:



Properties

- **ObjectId Id** [get, set]

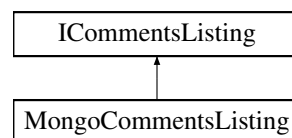
The documentation for this class was generated from the following file:

- MyMusicTaste/Models/Model.cs

5.26 MongoCommentsListing Class Reference

Provides methods for retrieving comments from the [MongoDb](#) database.

Inheritance diagram for MongoCommentsListing:



Public Member Functions

- Task< IEnumerable< [Comment](#) > > [GetCommentsByUserAsync](#) (string userId, int resultCount)
Retrieves comments made by a specific user.
- async Task< IEnumerable< [Comment](#) > > [GetCommentsByPageAsync](#) ([CommentPageType](#) pageType, string pageId, int resultCount)
Retrieves comments associated with a specific page, ordered by creation time descending.

5.26.1 Member Function Documentation

5.26.1.1 GetCommentsByPageAsync()

```

async Task< IEnumerable< Comment > > GetCommentsByPageAsync (
    CommentPageType pageType,
    string pageId,
    int resultCount) [inline]
  
```

Parameters

<i>pageType</i>	The type of the page the comments belong to.
<i>pageId</i>	The ID of the page.
<i>resultCount</i>	The maximum number of comments to return.

Returns

A task for the collection of comments.

Implements [ICommentsListing](#).

5.26.1.2 GetCommentsByUserAsync()

```
Task< IEnumerable< Comment > > GetCommentsByUserAsync (
    string userId,
    int resultCount) [inline]
```

Parameters

<i>userId</i>	The ID of the user whose comments to retrieve.
<i>resultCount</i>	The maximum number of comments to return.

Returns

A task for the list of comments.

Implements [ICommentsListing](#).

The documentation for this class was generated from the following file:

- MyMusicTaste/Database/Contexts/MongoDb/Operations/MongoCommentsListing.cs

5.27 MongoDBContext Class Reference

Provides a MongoDB client and a method to establish a connection.

Static Public Member Functions

- static void [Connect](#) (string? key)
Connects to the MongoDB database using the provided connection string.

Properties

- static ? MongoClient **Client** [get]
The singleton MongoDB client instance.

5.27.1 Member Function Documentation**5.27.1.1 Connect()**

```
void Connect (
    string? key) [inline], [static]
```


Parameters

key	The MongoDB connection string.
-----	--------------------------------

Exceptions

<i>ArgumentException</i>	Thrown if the connection string is null or empty.
--------------------------	---

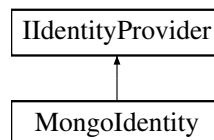
The documentation for this class was generated from the following file:

- MyMusicTaste/Database/Contexts/MongoDb/MongoDbContext.cs

5.28 Mongoidentity Class Reference

A MongoDB-based implementation of [IIdentityProvider](#) using ASP.NET Identity. Handles user signup, login, logout, and authentication/authorization checks.

Inheritance diagram for Mongoidentity:



Public Member Functions

- [Mongoidentity](#) (SignInManager< [MongoAccount](#) > signInManager, IDbRepository< [User](#) > userRepository, IHttpContextAccessor httpContextAccessor)
Initializes a new instance of [Mongoidentity](#) with required ASP.NET Identity services.
- async Task< IdentityResult > [SignUpUserAsync](#) ([IUserSignupDto](#) newUserSignup)
Signs up a new user and stores their account in MongoDB.
- async Task< SignInResult > [LoginUserAsync](#) ([IUserLoginDto](#) userLogin)
Logs in a user using ASP.NET Identity sign-in manager.
- async Task [LogOutUserAsync](#) ()
Logs out the currently signed-in user.
- Task< IdentityResult > [AssignRoleAsync](#) (IdentityUser user, IdentityRole role)
Assigns a role to a user. (Not implemented in this MongoDB version.)
- bool [IsAuthenticated](#) ()
Checks if the current HTTP context has an authenticated user.
- bool [AuthorizeUserById](#) (string requiredUserId)
Checks if the current user has a specific user ID claim.
- string? [GetUserId](#) ()
Retrieves the current user's ID from the HTTP context.

Static Public Member Functions

- static void [Configure](#) (IServiceCollection services, string connectionString)
Configures ASP.NET Identity services and registers [MongoIdentity](#) as the identity provider.

5.28.1 Constructor & Destructor Documentation

5.28.1.1 MongoIdentity()

```
MongoIdentity (
    SignInManager< MongoAccount > signInManager,
    IRepository< User > userRepository,
    IHttpContextAccessor httpContextAccessor) [inline]
```

Parameters

<i>signInManager</i>	The SignInManager for user login/logout.
<i>userRepository</i>	Repository for storing application user data.
<i>httpContextAccessor</i>	Accessor for the current HTTP context.

5.28.2 Member Function Documentation

5.28.2.1 AssignRoleAsync()

```
Task< IdentityResult > AssignRoleAsync (
    IdentityUser user,
    IdentityRole role) [inline]
```

Parameters

<i>user</i>	The identity user to assign the role to.
<i>role</i>	The identity role to assign.

Returns

A task for the IdentityResult.

Implements [IIdentityProvider](#).

5.28.2.2 AuthorizeUserById()

```
bool AuthorizeUserById (
    string requiredUserId) [inline]
```

Parameters

<i>required</i> ↔ <i>UserId</i>	The required user ID.
------------------------------------	-----------------------

Returns

True if the user has the claim; otherwise false.

Implements [IIdentityProvider](#).

5.28.2.3 Configure()

```
void Configure (
    IServiceCollection services,
    string connectionString) [inline], [static]
```

Parameters

<i>services</i>	The service collection to configure.
<i>connectionString</i>	The MongoDB connection string for the identity store.

5.28.2.4 GetUserId()

```
string? GetUserId () [inline]
```

Returns

The user ID as a string, or null if not authenticated.

Implements [IIdentityProvider](#).

5.28.2.5 IsAuthenticated()

```
bool IsAuthenticated () [inline]
```

Returns

True if a user is authenticated; otherwise false.

Implements [IIdentityProvider](#).

5.28.2.6 LoginUserAsync()

```
async Task< SignInResult > LoginUserAsync (
    IUserLoginDto userLogin) [inline]
```

Parameters

<i>userLogin</i>	The user login data transfer object.
------------------	--------------------------------------

Returns

A task for the SignInResult indicating success or failure.

Implements [IIdentityProvider](#).

5.28.2.7 LogOutUserAsync()

```
async Task LogOutUserAsync () [inline]
```

Implements [IIdentityProvider](#).

5.28.2.8 SignUpUserAsync()

```
async Task< IdentityResult > SignUpUserAsync (
    IUserSignupDto newUserSignup) [inline]
```

Parameters

<i>newUserSignup</i>	The user signup data transfer object.
----------------------	---------------------------------------

Returns

A task for the IdentityResult indicating success or failure.

Implements [IIdentityProvider](#).

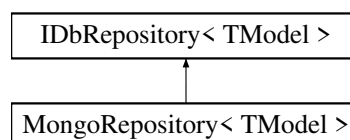
The documentation for this class was generated from the following file:

- MyMusicTaste/Database/Contexts/MongoDb/Mongolidentity.cs

5.29 MongoDBRepository< TModel > Class Template Reference

A MongoDB-based repository for performing CRUD operations on models.

Inheritance diagram for MongoDBRepository< TModel >:



Public Member Functions

- TModel [GetById](#) (string? id)
Retrieves a model by its ID.
- TModel [GetById](#) (ObjectId id)
Retrieves a model by its MongoDB ObjectId.
- async Task< TModel > [GetByIdAsync](#) (string? id)
Asynchronously retrieves a model by its ID.
- async Task< IEnumerable< TModel > > [GetByIdsAsync](#) (IEnumerable< string > ids)
Asynchronously retrieves multiple models by their IDs.
- async Task [CreateAsync](#) (TModel model)
Asynchronously inserts a new model into the database collection.
- async Task [UpdateAsync](#) (TModel model)
Asynchronously updates an existing model in the database collection.
- async Task [DeleteAsync](#) (TModel model)
Asynchronously deletes a model from the collection.

Properties

- IMongoCollection< TModel > **Collection** = MongoCollectionFactory.Create<TModel>() [get]
The MongoDB collection associated with the model type.

5.29.1 Detailed Description

Template Parameters

<i>TModel</i>	The type of model managed by this repository. Must inherit from Model .
---------------	---

Type Constraints

TModel : Model

5.29.2 Member Function Documentation

5.29.2.1 CreateAsync()

```
async Task CreateAsync (
    TModel model) [inline]
```

Parameters

<i>model</i>	The model to insert.
--------------	----------------------

Implements [IDbRepository< TModel >](#).

5.29.2.2 DeleteAsync()

```
async Task DeleteAsync (
    TModel model) [inline]
```

Parameters

<i>model</i>	The model to delete.
--------------	----------------------

Implements [IDbRepository< TModel >](#).

5.29.2.3 GetById() [1/2]

```
TModel GetById (
    ObjectId id) [inline]
```

Parameters

<i>id</i>	The MongoDB ObjectId of the model.
-----------	------------------------------------

Returns

The model corresponding to the specified ID.

Exceptions

<i>EntryNotFoundException</i>	Thrown when no entry is found.
-------------------------------	--------------------------------

Implements [IDbRepository< TModel >](#).

5.29.2.4 GetById() [2/2]

```
TModel GetById (
    string? id) [inline]
```

Parameters

<i>id</i>	The string representation of the model's ObjectId.
-----------	--

Returns

The model corresponding to the specified ID.

Exceptions

<i>EntryNotFoundException</i>	Thrown when the ID is invalid or no entry is found.
-------------------------------	---

Implements [IDbRepository< TModel >](#).

5.29.2.5 GetByIdAsync()

```
async Task< TModel > GetByIdAsync (
    string? id) [inline]
```

Parameters

<i>id</i>	The string representation of the model's ObjectId.
-----------	--

Returns

A task for the model corresponding to the specified ID.

Exceptions

<i>EntryNotFoundException</i>	Thrown when no entry is found.
-------------------------------	--------------------------------

Implements [IDbRepository< TModel >](#).

5.29.2.6 GetByIdsAsync()

```
async Task< IEnumerable< TModel > > GetByIdsAsync (  
    IEnumerable< string > ids) [inline]
```

Parameters

<i>ids</i>	The collection of string IDs to retrieve.
------------	---

Returns

A task for the collection of matching models.

Implements [IDbRepository< TModel >](#).

5.29.2.7 UpdateAsync()

```
async Task UpdateAsync (  
    TModel model) [inline]
```

Parameters

<i>model</i>	The model with updated values.
--------------	--------------------------------

Implements [IDbRepository< TModel >](#).

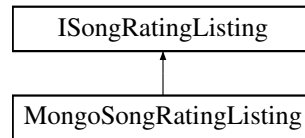
The documentation for this class was generated from the following file:

- MyMusicTaste/Database/Contexts/MongoDb/MongoRepository.cs

5.30 MongoSongRatingListing Class Reference

Retrieves song ratings from a MongoDB collection.

Inheritance diagram for MongoSongRatingListing:



Public Member Functions

- Task< [SongRating](#) > [GetSongRatingAsync](#) (string songId, string userId)
Retrieves the rating a specific user has given to a specific song.
- async Task< IEnumerable< [SongRating](#) > > [GetRatingsByUserAsync](#) (string userId)
Retrieves all ratings submitted by a specific user.
- async Task< IEnumerable< [SongRating](#) > > [GetRatingsBySongAsync](#) (string songId)
Retrieves all ratings associated with a specific song.

5.30.1 Member Function Documentation

5.30.1.1 GetRatingsBySongAsync()

```
async Task< IEnumerable< SongRating > > GetRatingsBySongAsync (
    string songId) [inline]
```

Parameters

<i>songId</i>	The ID of the song.
---------------	---------------------

Returns

A task for the collection of ratings.

Implements [ISongRatingListing](#).

5.30.1.2 GetRatingsByUserAsync()

```
async Task< IEnumerable< SongRating > > GetRatingsByUserAsync (
    string userId) [inline]
```


Parameters

<i>userId</i>	The ID of the user.
---------------	---------------------

Returns

A task for the collection of ratings.

Implements [ISongRatingListing](#).

5.30.1.3 GetSongRatingAsync()

```
Task< SongRating > GetSongRatingAsync (
    string songId,
    string userId) [inline]
```

Parameters

<i>songId</i>	The ID of the song.
<i>userId</i>	The ID of the user.

Returns

A task for the retrieved song rating, or null if no rating exists.

Implements [ISongRatingListing](#).

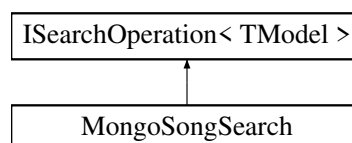
The documentation for this class was generated from the following file:

- MyMusicTaste/Database/Contexts/MongoDb/Operations/MongoSongRatingListing.cs

5.31 MongoSongSearch Class Reference

Performs search operations for users in MongoDB.

Inheritance diagram for MongoSongSearch:



Public Member Functions

- async Task< IEnumerable< [Song](#) >? > [SearchAsync](#) (string query, int resultsCount)
Searches users by username using autocomplete.

5.31.1 Member Function Documentation

5.31.1.1 SearchAsync()

```
async Task< IEnumerable< Song >? > SearchAsync (
    string query,
    int resultsCount) [inline]
```

Parameters

<i>query</i>	The search query string.
<i>resultsCount</i>	The maximum number of results to return.

Returns

A task for the collection of matching users, or null if the query is empty.

Implements [ISearchOperation< TModel >](#).

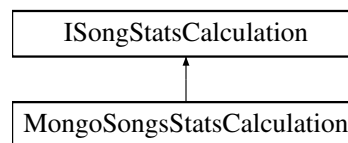
The documentation for this class was generated from the following file:

- MyMusicTaste/Database/Contexts/MongoDb/Operations/MongoSongSearch.cs

5.32 MongoSongsStatsCalculation Class Reference

Calculates statistics for a song, including average rating, median rating, total listens, and rating distribution in MongoDB.

Inheritance diagram for MongoSongsStatsCalculation:

**Public Member Functions**

- async Task< [SongStats](#) > [CalculateSongStatsAsync](#) ([Song](#) song)
Calculates statistics for a song, including average rating, median rating, total listens, and rating distribution.

5.32.1 Member Function Documentation

5.32.1.1 CalculateSongStatsAsync()

```
async Task< SongStats > CalculateSongStatsAsync (  
    Song song) [inline]
```

Parameters

<i>song</i>	The song to calculate statistics for.
-------------	---------------------------------------

Returns

A task for the song's statistics.

Implements [ISongStatsCalculation](#).

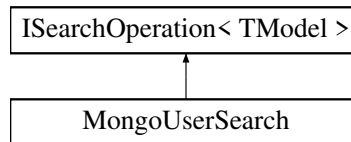
The documentation for this class was generated from the following file:

- MyMusicTaste/Database/Contexts/MongoDb/Operations/MongoSongsStatsCalculation.cs

5.33 MongoUserSearch Class Reference

Performs search operations for songs in MongoDB.

Inheritance diagram for MongoUserSearch:



Public Member Functions

- `async Task< IEnumerable< User >?> SearchAsync` (string query, int resultsCount)
Searches songs by title using autocomplete.

5.33.1 Member Function Documentation

5.33.1.1 SearchAsync()

```

async Task< IEnumerable< User >?> SearchAsync (
    string query,
    int resultsCount) [inline]
  
```

Parameters

<i>query</i>	The search query string.
<i>resultsCount</i>	The maximum number of results to return.

Returns

A task for the collection of matching songs, or null if the query is empty.

Implements [ISearchOperation< TModel >](#).

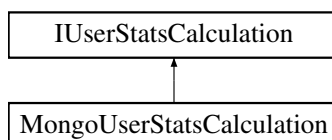
The documentation for this class was generated from the following file:

- `MyMusicTaste/Database/Contexts/MongoDb/Operations/MongoUserSearch.cs`

5.34 MongoUserStatsCalculation Class Reference

Calculates statistics for a user based on their song ratings in MongoDB.

Inheritance diagram for MongoUserStatsCalculation:



Public Member Functions

- `async Task< UserStats > CalculateUserStatsAsync (string userId)`
Calculates statistics for a user based on their ratings, including average ratings by album, author, and genre.

5.34.1 Member Function Documentation

5.34.1.1 CalculateUserStatsAsync()

```
async Task< UserStats > CalculateUserStatsAsync (
    string userId) [inline]
```

Parameters

<i>userId</i>	The ID of the user to calculate statistics for.
---------------	---

Returns

A task for the user's statistics.

Implements [IUserStatsCalculation](#).

The documentation for this class was generated from the following file:

- MyMusicTaste/Database/Contexts/MongoDb/Operations/MongoUserStatsCalculation.cs

5.35 RateSongComp Class Reference

A component that allows a signed-in user to rate a song, view their existing rating, update it, or remove it from their collection.

Protected Member Functions

- override `async Task OnInitializedAsync ()`

Properties

- string **SongId** = null! [get, set]
The ID of the song to rate.

The documentation for this class was generated from the following file:

- MyMusicTaste/Components/Page-Song/RateSongComp.razor.cs

5.36 SearchComponent< TModel > Class Template Reference

Base component for search pages. Provides shared search functionality for a given model type.

Public Member Functions

- async Task **UpdateResults** ()
Performs the search with the current query and updates the [Results](#).

Protected Member Functions

- override async Task **OnAfterRenderAsync** (bool _)

Properties

- string? **Query** [get, set]
The search query string entered by the user.
- int **ResultsCount** [get, set]
Maximum number of results to return.
- ISearchOperation< TModel > **Searcher** = null! [get, set]
The search operation service used to perform the query.
- IEnumerable< TModel >? **Results** [get, set]
The list of search results returned by the query.

5.36.1 Detailed Description

Template Parameters

<i>TModel</i>	The type of the model to search for, must inherit from Model .
---------------	--

Type Constraints

TModel* : *Model

The documentation for this class was generated from the following file:

- MyMusicTaste/Components/Page-Search/SearchComponent.cs

5.37 SearchPage Class Reference

A page providing a search interface for songs and users with tabbed navigation.

Static Public Member Functions

- static string **GetRoute** ()

Static Public Attributes

- const string **ROUTE_TEMPLATE** = "/search"

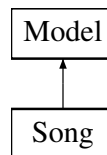
The documentation for this class was generated from the following file:

- MyMusicTaste/Components/Page-Search/SearchPage.razor.cs

5.38 Song Class Reference

A model representing basic song information.

Inheritance diagram for Song:



Properties

- string **Title** = null! [get, set]
- string **Author** = null! [get, set]
- string? **Album** [get, set]
- string? **Genre** [get, set]
- DateOnly **ReleaseDate** = INVALID_DATE [get, set]
- bool **ReleasDateValid** [get]
- string? **CoverImageLink** [get, set]
- string? **SourceLink** [get, set]

Properties inherited from [Model](#)

- ObjectId **Id** [get, set]

The documentation for this class was generated from the following file:

- MyMusicTaste/Models/Song/Song.cs

5.39 SongPage Class Reference

DPage for displaying detailed information about a song, including its metadata, statistics, and comments.

Static Public Member Functions

- static string **GetRoute** (string songId)

Static Public Attributes

- const string **ROUTE_TEMPLATE** = "/songs/{SongId}"

Protected Member Functions

- override async Task **OnInitializedAsync** ()

Properties

- string **SongId** = null! [get, set]
The ID of the song to display.

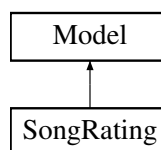
The documentation for this class was generated from the following file:

- MyMusicTaste/Components/Page-Song/SongPage.razor.cs

5.40 SongRating Class Reference

Represents a rating given by a user to a song.

Inheritance diagram for SongRating:



Static Public Attributes

- const byte **NOT_RATED** = 255
Sentinel value of the Rating field, indicating that a song has not been rated.

Properties

- ObjectId **UserId** [get, set]
- ObjectId **SongId** [get, set]
- byte **Rating** = **NOT_RATED** [get, set]
- bool **IsRated** [get]

Properties inherited from **Model**

- ObjectId **Id** [get, set]

The documentation for this class was generated from the following file:

- MyMusicTaste/Models/User/SongRating.cs

5.41 SongRatingItem Class Reference

Displays a single song rating item for a user. Can optionally allow inline editing of the rating.

Protected Member Functions

- override void **OnInitialized** ()

Properties

- [SongRating](#) **Rating** = new() [get, set]
The rating object of the item.
- bool **Editable** [get, set]
Determines whether the rating can be edited inline.
- EventCallback **OnRatingSaved** [get, set]
Invoked after a rating has been saved.

The documentation for this class was generated from the following file:

- MyMusicTaste/Components/Page-User/SongRatingItem.razor.cs

5.42 SongRatingSection Class Reference

Displays a list of a user's song ratings in descending order. Allows drag-and-drop to reorder ratings if the user is authorized.

Protected Member Functions

- override async Task **OnInitializedAsync** ()

Properties

- string **UserId** = null! [get, set]
The ID of the user whose ratings will be displayed.

The documentation for this class was generated from the following file:

- MyMusicTaste/Components/Page-User/SongRatingSection.razor.cs

5.43 SongSearchItem Class Reference

Displays an item in search results that navigates to the song's page when clicked.

Properties

- Models.? Song **Song** = new() [get, set]

The documentation for this class was generated from the following file:

- MyMusicTaste/Components/Page-Search/SongSearchItem.razor.cs

5.44 SongStats Class Reference

Represents statistical information about a song, including ratings and listens.

Properties

- int **TotalListens** [get, set]
Total number of times the song has been rated.
- float **AverageRating** [get, set]
- float **MedianRating** [get, set]
- int[] **RatingDistribution** = new int[20] [get, set]
Distribution of ratings across 20 buckets, grouped by 1-5, 6-10, etc.
- bool **NoData** [get]
True if no ratings of the song exist.

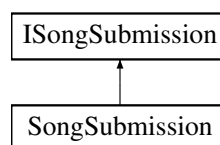
The documentation for this class was generated from the following file:

- MyMusicTaste/Models/Song/SongStats.cs

5.45 SongSubmission Class Reference

Handles submission of new songs to the MongoDB database, ensuring no duplicates exist.

Inheritance diagram for SongSubmission:



Public Member Functions

- async Task< string > [SubmitSongAsync](#) (Song song)
Submits a new song to the database.

5.45.1 Member Function Documentation

5.45.1.1 SubmitSongAsync()

```

async Task< string > SubmitSongAsync (
    Song song) [inline]
  
```

Parameters

<i>song</i>	The song to submit.
-------------	---------------------

Returns

A task for the ID of the newly created song as a string.

Exceptions

<i>EntryAlreadyExistsException</i>	Thrown if the song already exists in the repository.
------------------------------------	--

Implements [ISongSubmission](#).

The documentation for this class was generated from the following file:

- MyMusicTaste/Database/Contexts/MongoDb/Operations/MongoSongSubmission.cs

5.46 SongSubmissionPage Class Reference

Page for submitting a new song to the database. Validates user input and handles navigation after submission.

Static Public Member Functions

- static string **GetRoute** ()

Static Public Attributes

- const string **ROUTE_TEMPLATE** = "/submit-song"

Protected Member Functions

- override void **OnInitialized** ()

The documentation for this class was generated from the following file:

- MyMusicTaste/Components/Page-SongSubmission/SongSubmissionPage.razor.cs

5.47 Thumbnail Class Reference

A component that displays an image thumbnail or a placeholder if the image link is invalid.

Protected Member Functions

- override async Task **OnParametersSetAsync** ()

Properties

- string? **ImageLink** [get, set]
The URL of the image to display in the thumbnail.
- string? **AltText** [get, set]
Alternative text to display if the image cannot be shown.
- Dictionary< string, object > **UnmatchedAttributes** = null! [get, set]
Additional HTML attributes to apply to the image or placeholder container.

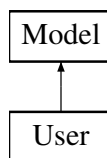
The documentation for this class was generated from the following file:

- MyMusicTaste/Components/Misc/Thumbnail.razor.cs

5.48 User Class Reference

A model representing basic user information.

Inheritance diagram for User:



Properties

- required string **Username** [get, set]
- string? **AboutMe** [get, set]
- string? **ProfilePictureLink** [get, set]
- string? **BannerPictureLink** [get, set]

Properties inherited from **Model**

- ObjectId **Id** [get, set]

The documentation for this class was generated from the following file:

- MyMusicTaste/Models/User/User.cs

5.49 UserPage Class Reference

Displays a user's profile including general info, statistics, and comments. Supports editing the profile for the authorized user.

Static Public Member Functions

- static string **GetRoute** (ObjectId userId)

Static Public Attributes

- const string **ROUTE_TEMPLATE** = "/users/{[UserId](#)}"

Protected Member Functions

- override async Task **OnInitializedAsync** ()

Properties

- string **UserId** = null! [get, set]
The ID of the user whose profile is being displayed.

The documentation for this class was generated from the following file:

- MyMusicTaste/Components/Page-User/UserPage.razor.cs

5.50 UserSearchItem Class Reference

Displays an item in search results that navigates to the user's page when clicked.

Properties

- Models.? User **User** [get, set]
The user data to display.

The documentation for this class was generated from the following file:

- MyMusicTaste/Components/Page-Search/UserSearchItem.razor.cs

5.51 UserSignupForm Class Reference

A form component that allows new users to sign up for the application. Handles input validation, submission, and automatic login after successful registration.

The documentation for this class was generated from the following file:

- MyMusicTaste/Components/Page-Auth/UserSignupForm.razor.cs

5.52 UserStats Class Reference

Represents statistical information about a user's ratings.

Properties

- List<(string, double)>? **AlbumsByMean** [get, set]
Average ratings per album, as a list of (album name, mean rating) tuples.
- List<(string, double)>? **AuthorsByMean** [get, set]
Average ratings per author, as a list of (author name, mean rating) tuples.
- List<(string, double)>? **GenresByMean** [get, set]
Average ratings per genre, as a list of (genre name, mean rating) tuples.

The documentation for this class was generated from the following file:

- MyMusicTaste/Models/User/UserStats.cs

Index

- AssignRoleAsync
 - IIdentityProvider, [28](#)
 - Mongolidentity, [42](#)
- AuthController, [15](#)
 - LoginAsync, [15](#)
 - LogoutAsync, [16](#)
- AuthController.LoginDto, [37](#)
 - Password, [38](#)
 - Username, [38](#)
- AuthorizeUserById
 - IIdentityProvider, [29](#)
 - Mongolidentity, [42](#)
- CalculateSongStatsAsync
 - ISongStatsCalculation, [34](#)
 - MongoSongsStatsCalculation, [50](#)
- CalculateUserStatsAsync
 - IUserStatsCalculation, [37](#)
 - MongoUserStatsCalculation, [52](#)
- Comment, [16](#)
- CommentComp, [17](#)
- CommentSection, [17](#)
- Configure
 - Mongolidentity, [43](#)
- ConfirmDialog, [18](#)
- Connect
 - MongoDbContext, [40](#)
- CreateAsync
 - IDbRepository< TModel >, [26](#)
 - MongoRepository< TModel >, [45](#)
- CssSize
 - MyMusicTaste.Utils, [13](#)
- DatabaseOperationException, [19](#)
- DeleteAsync
 - IDbRepository< TModel >, [26](#)
 - MongoRepository< TModel >, [45](#)
- Dialog< TResult >, [20](#)
 - OpenDialog, [21](#)
- DialogBase, [21](#)
- DialogResult< TResult >, [22](#)
- Dropzone, [22](#)
 - SetActive, [23](#)
- GetById
 - IDbRepository< TModel >, [26](#)
 - MongoRepository< TModel >, [46](#)
- GetByIdAsync
 - IDbRepository< TModel >, [27](#)
 - MongoRepository< TModel >, [46](#)
- GetByIdsAsync
 - IDbRepository< TModel >, [27](#)
 - MongoRepository< TModel >, [47](#)
- GetCommentsByPageAsync
 - ICommentsListing, [24](#)
 - MongoCommentsListing, [39](#)
- GetCommentsByUserAsync
 - ICommentsListing, [24](#)
 - MongoCommentsListing, [40](#)
- GetRatingsBySongAsync
 - ISongRatingListing, [33](#)
 - MongoSongRatingListing, [48](#)
- GetRatingsByUserAsync
 - ISongRatingListing, [33](#)
 - MongoSongRatingListing, [48](#)
- GetSongRatingAsync
 - ISongRatingListing, [34](#)
 - MongoSongRatingListing, [49](#)
- GetUserId
 - IIdentityProvider, [29](#)
 - Mongolidentity, [43](#)
- Histogram< TValue, TLabel >, [23](#)
- ICommentsListing, [24](#)
 - GetCommentsByPageAsync, [24](#)
 - GetCommentsByUserAsync, [24](#)
- IDbRepository< TModel >, [25](#)
 - CreateAsync, [26](#)
 - DeleteAsync, [26](#)
 - GetById, [26](#)
 - GetByIdAsync, [27](#)
 - GetByIdsAsync, [27](#)
 - UpdateAsync, [27](#)
- IIdentityProvider, [28](#)
 - AssignRoleAsync, [28](#)
 - AuthorizeUserById, [29](#)
 - GetUserId, [29](#)
 - IsAuthenticated, [29](#)
 - LoginUserAsync, [29](#)
 - LogOutUserAsync, [30](#)
 - SignUpUserAsync, [30](#)
- InputDialog, [30](#)
- IsAuthenticated
 - IIdentityProvider, [29](#)
 - Mongolidentity, [43](#)
- ISearchOperation< TModel >, [32](#)
 - SearchAsync, [32](#)
- ISongRatingListing, [33](#)
 - GetRatingsBySongAsync, [33](#)

- GetRatingsByUserAsync, 33
 - GetSongRatingAsync, 34
- ISongStatsCalculation, 34
 - CalculateSongStatsAsync, 34
- ISongSubmission, 35
 - SubmitSongAsync, 35
- IUserLoginDto, 36
- IUserSignupDto, 36
- IUserStatsCalculation, 37
 - CalculateUserStatsAsync, 37
- LoginAsync
 - AuthController, 15
- LoginPage, 38
- LoginUserAsync
 - IIdentityProvider, 29
 - Mongolidentity, 43
- LogoutAsync
 - AuthController, 16
- LogOutUserAsync
 - IIdentityProvider, 30
 - Mongolidentity, 44
- Model, 39
- MongoCommentsListing, 39
 - GetCommentsByPageAsync, 39
 - GetCommentsByUserAsync, 40
- MongoDbContext, 40
 - Connect, 40
- Mongolidentity, 41
 - AssignRoleAsync, 42
 - AuthorizeUserById, 42
 - Configure, 43
 - GetUserId, 43
 - IsAuthenticated, 43
 - LoginUserAsync, 43
 - LogOutUserAsync, 44
 - Mongolidentity, 42
 - SignUpUserAsync, 44
- MongoRepository< TModel >, 44
 - CreateAsync, 45
 - DeleteAsync, 45
 - GetById, 46
 - GetByIdAsync, 46
 - GetByIdsAsync, 47
 - UpdateAsync, 47
- MongoSongRatingListing, 48
 - GetRatingsBySongAsync, 48
 - GetRatingsByUserAsync, 48
 - GetSongRatingAsync, 49
- MongoSongSearch, 49
 - SearchAsync, 49
- MongoSongsStatsCalculation, 50
 - CalculateSongStatsAsync, 50
- MongoUserSearch, 51
 - SearchAsync, 51
- MongoUserStatsCalculation, 51
 - CalculateUserStatsAsync, 52
- MyMusicTaste, 9
 - MyMusicTaste.Components, 9
 - MyMusicTaste.Components.Comments, 9
 - MyMusicTaste.Components.Dialogs, 9
 - MyMusicTaste.Components.Misc, 10
 - MyMusicTaste.Components.Page_Auth, 10
 - MyMusicTaste.Components.Page_Home, 10
 - MyMusicTaste.Components.Page_Search, 10
 - MyMusicTaste.Components.Page_Song, 10
 - MyMusicTaste.Components.Page_SongSubmission, 11
 - MyMusicTaste.Components.Page_User, 11
 - MyMusicTaste.Database, 11
 - MyMusicTaste.Database.Contexts, 11
 - MyMusicTaste.Database.Contexts.MongoDb, 11
 - MyMusicTaste.Database.Contexts.MongoDb.Operations, 12
 - MyMusicTaste.Database.Operations, 12
 - MyMusicTaste.Models, 13
 - MyMusicTaste.Startup, 13
 - MyMusicTaste.Utils, 13
 - CssSize, 13
- OpenDialog
 - Dialog< TResult >, 21
- Password
 - AuthController.LoginDto, 38
- RateSongComp, 52
- SearchAsync
 - ISearchOperation< TModel >, 32
 - MongoSongSearch, 49
 - MongoUserSearch, 51
- SearchComponent< TModel >, 53
- SearchPage, 53
- SetActive
 - Dropzone, 23
- SignUpUserAsync
 - IIdentityProvider, 30
 - Mongolidentity, 44
- Song, 54
- SongPage, 54
- SongRating, 55
- SongRatingItem, 56
- SongRatingSection, 56
- SongSearchItem, 56
- SongStats, 57
- SongSubmission, 57
 - SubmitSongAsync, 57
- SongSubmissionPage, 58
- SubmitSongAsync
 - ISongSubmission, 35
 - SongSubmission, 57
- Thumbnail, 58
- UpdateAsync
 - IDbRepository< TModel >, 27
 - MongoRepository< TModel >, 47
- User, 59

Username
 AuthController.LoginDto, [38](#)
UserPage, [60](#)
UserSearchItem, [60](#)
UserSignupForm, [61](#)
UserStats, [61](#)